THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 23

Delta Live Tables — Now Lakeflow Pipelines

SECTION 01

A Name Change, Not a Rebuild

CURRENT STATUS

The product formerly known as Delta Live Tables (DLT) has been updated to Lakeflow pipelines. If you have previously used DLT, there is no migration required: your existing code still runs. Along the way it was briefly called Lakeflow Declarative Pipelines and Lakeflow Spark Declarative Pipelines; expect to see all of these names in job postings, docs, and interviews, referring to the same underlying framework.

Databricks folded Delta Live Tables into Lakeflow, its unified data engineering product, and also contributed the underlying engine to the Apache Spark project as Spark Declarative Pipelines. The core mental model, streaming tables and materialized views connected in a dependency graph, has not changed across any of these renames.

SECTION 02

What the Framework Actually Does

This is a declarative framework for building batch and streaming data pipelines in SQL and Python. Instead of writing imperative orchestration code, you describe each table as a query, and the runtime figures out dependency resolution, incremental processing, orchestration, retries, and data quality enforcement automatically.

- You define what the final table should contain; the engine determines the execution order and refresh strategy.
- It runs on the performance-optimized Databricks Runtime, and the underlying flows API uses the same DataFrame API as Apache Spark and Structured Streaming.
- Common use cases include incremental ingestion from cloud storage or message buses, incremental batch and streaming transformations, and real-time processing between transactional stores.

SECTION 03

The Three Core Building Blocks

Type	Behavior	Best For
Streaming Table	A Delta table with additional support for streaming, incremental data processing	Continuously arriving data, such as Bronze ingestion
Materialized View	Precomputed results stored physically, refreshed incrementally as source data changes	Expensive aggregations queried frequently
View	Computed fresh on every query, not materialized	Lightweight logic that doesn't need to be persisted

SECTION 04

Data Quality Expectations

Expectations let you declare constraints directly as part of a table definition, and the pipeline can enforce them, drop violating rows, or quarantine bad data, without needing separate validation code.

```
import dlt

@dlt.table
@dlt.expect("valid_amount", "amount > 0")
@dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL")
@dlt.expect_or_fail("valid_currency", "currency IN ('USD','INR','EUR')")
def silver_transactions():
    return spark.readStream.table("bronze_transactions")
```

Expectation Type	Behavior on Violation
expect	Record kept, violation logged as a metric
expect_or_drop	Violating record silently dropped from output
expect_or_fail	Pipeline update fails immediately

SECTION 05

Handling Change Data Capture

For upsert-style CDC feeds, the framework provides a dedicated syntax rather than requiring hand-written MERGE logic.

```
-- SQL: AUTO CDC INTO (also seen as APPLY CHANGES INTO in older docs)
CREATE OR REFRESH STREAMING TABLE silver_customers;

CREATE FLOW customers_cdc_flow AS AUTO CDC INTO
  silver_customers
FROM STREAM(bronze_customers_cdc)
KEYS (customer_id)
SEQUENCE BY updated_at
STORED AS SCD TYPE 2;
```

This single declaration handles deduplication, out-of-order events, and even Slowly Changing Dimension Type 2 history tracking, which would otherwise require significant hand-written merge logic.

SECTION 06

The Declarative Dependency Graph

```
You declare:
  bronze_table  (streaming table, from cloudFiles)
       |
  silver_table  (materialized view, cleaned + expectations)
       |
  gold_table    (materialized view, aggregated)

The engine figures out:
  - execution order (via the dependency graph above)
  - what's new since the last run (incremental processing)
  - retries, checkpointing, and orchestration
  - which rows violate expectations, and what to do about it
```

REMEMBER THIS FOR INTERVIEWS

The defining trait of this framework, whatever name it goes by, is that you write what the table should be, not how to compute and orchestrate it. That's the core interview distinction between a declarative pipeline framework and a traditional, imperative ETL job.

SECTION 07

Glossary — Day 23 Terms

Term	Definition
Delta Live Tables (DLT)	The original name for Databricks' declarative pipeline framework; now renamed to Lakeflow pipelines.
Lakeflow Pipelines	The current name for the same declarative framework, following its move under the broader Lakeflow product umbrella.
Streaming Table	A Delta table with additional support for streaming or incremental data processing within a pipeline.
Materialized View	A precomputed, physically stored result set that refreshes incrementally as source data changes.
Expectations	Declared data quality constraints a pipeline can enforce, drop violating rows for, or fail the run on.
AUTO CDC INTO	The current syntax (formerly APPLY CHANGES INTO) for declaratively handling upsert-style change data capture, including SCD Type 2 history.

SECTION 08

Interview Questions & Answers

Q1. What happened to Delta Live Tables?

A. It was renamed to Lakeflow pipelines (having also briefly been called Lakeflow Declarative Pipelines), as part of Databricks folding it into its broader Lakeflow product. No migration is required; existing DLT code continues to run unchanged.

Q2. What does 'declarative' mean in this context?

A. You describe each target table as a query defining what it should contain, and the runtime automatically handles dependency resolution, incremental processing, orchestration, retries, and data quality enforcement, rather than you writing that orchestration logic by hand.

Q3. What is the difference between a streaming table and a materialized view in this framework?

A. A streaming table is a Delta table with additional support for continuous, incremental processing of new data. A materialized view stores a precomputed result that refreshes incrementally as source data changes, better suited to expensive, frequently queried aggregations.

Q4. What are Expectations, and what are the three ways they can handle a violation?

A. Expectations are declared data quality constraints on a table. A plain expect logs violations as a metric but keeps the record; expect_or_drop silently drops the violating record; expect_or_fail fails the entire pipeline update immediately.

Q5. How does this framework handle CDC without hand-written MERGE logic?

A. Through a dedicated syntax, currently AUTO CDC INTO (previously APPLY CHANGES INTO), which declaratively handles deduplication, out-of-order events, and even SCD Type 2 history tracking from a single statement.

Q6. If a job posting mentions Delta Live Tables and another mentions Lakeflow Declarative Pipelines, are they asking about different things?

A. No. They refer to the same underlying declarative pipeline framework across different points in its rename history; the core concepts, streaming tables, materialized views, and expectations, are identical.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab